# 交互、部署与接口调用

## 1、HTTP协议基础

- 请求方法(Method)：使用POST将数据放在请求体(Body)中
- 内容类型(Content-Type)：基础题使用`application/。x-www-form-unlencoded`或`multipart/form-data`传输文本表单。
- 状态码(Status Code)：200成功，400客户端传参错误，500服务器其内部崩溃，并在代码中通过`try...except`实现了精确的状态码控制。

## 2、Flask框架核心机制

- 路由(Route)：通过`@app.route('/calculate',methods=['POST'])`定义接口门牌号。
- 请求解析(Request)：`request.form.get('key')`专门用于从表单格式的请求体中提取数据。使用`.get()`而不是`request.form'key'`是为了防止键不存在时抛出`KeyError`导致服务崩溃。
- 响应构造(Response)：`jsonify({"result":result})`，自动将Python字典序列化为标准的JSON字符串，并自动在响应头里加上`Content-Type:application/json`让前端能够正确解析。

## 问题

- 状态码400-{"erroe":"缺少参数"}：在Apifox里做放崩溃测试时，将参数名Key当成了业务数据去修改，导致后端认不出这个参数。解决：Key必须是固定的变量名，而Value才是用户世纪传入的数据。
- 除数为0：传入`Operation=divide,Data_B=0`。解决：通过提前拦截`if data_b==0`，成功返回了错误信息的JSON，避免了后端的`ZeroDivisionError`崩溃。